

Dart: 進階語法(1)

定義類別關係

- 準備工作：
 - 將 dartex2 專案目錄，複製一份成 dartex3，做為開發專案資料夾。

```
cp -r dartex2 dartex3
```

- 複製之前，請注意目前所在的工作目錄
- Task 目標：
 - 實作文字介面指令 httpex，包含可使用參數、引數等可操作的完整指令。
 - 操作指令範例如下：

```
$ httpex help --verbose --command=search
```

- 可使用的參數名稱：
 - httpex: 可執行的指令名稱，與應用程式名稱相同。
 - help: 一種子指令，可查詢指令運作的方式與相關的參數使用方式。
 - -verbose: 一個旗標參數，可使指令詳細顯示運作的過程。
 - -command=search: 一個帶有資料值的參數，參數名稱是 command，而其資料值是 search。
- Task1: 定義引數層次結構
 - 新增一個檔案 command_runner/lib/src/arguments.dart，並且開啟該檔案。
 - 本檔案將定義 Argument、Option、Command、ArgResults 等類別。
 - 在該檔案內，定義一個 enum 資料型態的變數，名為 OptionType:

```
enum OptionType { flag, option }
```

- enum 為枚舉資料型態，可以是 flag 旗標 boolean 值，也可以是一般 option 項目值。
- 持續在該檔案內，定義一抽象類別，名為 Argument，處理各項參數，置於 enum 下方：

```
(前面省略...)
abstract class Argument {
  String get name;
  String? get help;

  Object? get defaultValue;
  String? get valueHelp;

  String get usage;
}
```

- name: 識別唯一名稱的參數。
 - help: 提供說明的參數名稱。

- `defaultValue`: `Object?` 型態的資料值，可以是 `bool` 布林值，也可以是字串資料值。
- `valueHelp`: 定義一個預期內的字串資料值。
- `usage`: 定義如何使用參數的顯示文字字串。
- `Argument` 為一抽象類別，其目的是為了方便子類別在繼承該類別之後，可輕易實作出參數的行為。
- 定義一個名為 `Option` 類別，繼承自 `Argument` 類別
 - `Option` 類別實現命令列中的選項參數，例：`--version` 或是 `--output=file.txt` 項目
 - 在 `command_runner/lib/src/arguments.dart` 檔案內，編寫下列程式內容：

```
class Option extends Argument {
  Option(
    this.name, {
      required this.type,
      this.help,
      this.abbr,
      this.defaultValue,
      this.valueHelp,
    });

  @override
  final String name;

  final OptionType type;

  @override
  final String? help;

  final String? abbr;

  @override
  final Object? defaultValue;

  @override
  final String? valueHelp;

  @override
  String get usage {
    if (abbr != null) {
      return '-$abbr,--$name: $help';
    }

    return '--$name: $help';
  }
}
```

- `extends` 為建立繼承關係的關鍵字
- 類別前使用的 `@override` 註解關鍵字，則是宣告其屬性與 `getter` 將取代 `Argument` 內的佔位符號。
- 上述程式中，增加的 `type` 為 `OptionType` 列舉型態，配合 `abbr` 項目，清楚交待 `usage` 這個 `getter` 的結構。
- 完成 `Option` 類別之後，將會產生一特別的參數資料型態。
 - 之後，只要定義 `Command` 類別，另一參數型態也會讓 CLI 應用程式執行。

- 定義一抽象類別(`abstract class`)，名為 `Command`，繼承自 `Argument`
 - `Command` 類別將重新展現可執行的運作，因為它提供其它指令可仿效的樣板

(以上省略...)

// 在 `command_runner/lib/src/arguments.dart` 檔案下，`Option` 類別之後
`abstract class Command extends Argument {`

`// 這空間先暫時保留一下`

`}`

- `abstract` 關鍵字來定義 `Command` 類別，表示該類別無法被直接實例化產生物件。
- 接下來，產生核心屬性。指令通常都需要名稱以及說明，執行時亦需要參考回到 `CommandRunner`。

// 在 `command_runner/lib/src/arguments.dart` 檔案下，`Command` 抽象類別內的程式
`abstract class Command extends Argument {`

`@override`

`String get name;`

`String get description;`

`bool get requiresArgument => false;`

`late CommandRunner runner;`

`@override`

`String? help;`

`@override`

`String? defaultValue;`

`@override`

`String? valueHelp;`

`}`

- `runner` 是 `CommandRunner` 型態，稍晚定義於 `command_runner_base.dart` 檔案內。
- `late` 關鍵字用於延遲初始化該變數值，但一定要在使用前宣告。
- 因為需要使用 `CommandRunner` 型態，是故在檔案最上方，需加入下列程式碼：

```
import '../command_runner.dart';
```

- 下一步，為了避免其它程式非預期地修改了選項參數，可以利用將參數值置入 `_private` 清單內。
 - Dart 程式語法只，只要在變數或是欄位名稱前，加上底線(`_`)符號，表示其為私有函式庫程式。
- `UnmodifiableSwetView` 表示透過不直接存取、唯讀且不可修改的視圖，來公開選項。
 - 這種方法是封裝的核心部分：限制對類別內部狀態的直接存取，以防止意外的干擾。
- `UnmodifiableSwetView` 類別是 Dart 核心集合函式庫，使用之前，必須引入該函式庫。

- 引入方式如下：

```
import 'dart:collection'; // 新增的一行
import '../command_runner.dart';
(以下省略....)
```

- 增加 options 清單以及 getter 收集器到 Command 類別：

```
abstract class Command extends Argument {
  // 保留已存在的屬性參數值

  @override
  String? valueHelp;

  // 新增下列幾行

  final List<Option> _options = [];

  UnmodifiableSetView<Option> get options =>
    UnmodifiableSetView(_options.toSet());
}
```

- 因為 _options 為私有變數名稱，外部程式無法直接操作該變數。
 - 為了存取 _options 變數，可另外新增兩個內部存取方法：addFlag 以及 addOption。
 - 這些方法將實例化 Option 物件，並且新增至私有清單內：

```
abstract class Command extends Argument {
  // 保留原屬性值與 getter 收集器

  UnmodifiableSetView<Option> get options =>
    UnmodifiableSetView(_options.toSet());

  // 新增下列幾行程式內容

  // 定義 Option 旗標的型態為布林值

  void addFlag(String name, {String? help, String? abbr, String? valueHelp,
    _options.add(
      Option(
        name,
        help: help,
        abbr: abbr,
        defaultValue: false,
        valueHelp: valueHelp,
        type: OptionType.flag,
      ),
    );
}
```

```
// Option 取得一內容值
void addOption(
  String name, {
    String? help,
    String? abbr,
    String? defaultValue,
    String? valueHelp,
  }) {
  _options.add(
    Option(
      name,
      help: help,
      abbr: abbr,
      defaultValue: defaultValue,
      valueHelp: valueHelp,
      type: OptionType.option,
    ),
  );
}
```

- 最後，每一個指令必須要有執行的邏輯順序。利用定義抽象 run 方法來具體執行。
- 因為一個指令可能是同步或是非同步執行，所以 run 方法從 dart:async 回傳 FutureOr 型態值。
 - 這裡的回傳值可以是原生值或是 Future 類別型態。
- 更新檔案開頭的引入項目內容：

```
import 'dart:async'; // 新引入的項目
import 'dart:collection';
import '../command_runner.dart';

(以下省略....)
```

- 接下來，可新增抽象 run 方法，以及提供 usage 項目，完成 Command 類別：

```
abstract class Command extends Argument {
  // 保留原屬性值、getter 收集器，以及其它的方法函式

  void addOption(
    String name, {
      String? help,
      String? abbr,
      String? defaultValue,
      String? valueHelp,
    }) {
    _options.add(
      Option(
        name,
        help: help,
        abbr: abbr,
```

```

        defaultValue: defaultValue,
        valueHelp: valueHelp,
        type: OptionType.option,
      ),
    );
}

// 增加下列幾行程式內容

FutureOr<Object?> run(ArgResults args);

@override
String get usage {
  return '$name: $description';
}
}

```

- `run(ArgResults args)`: 這個抽象方法本身是邏輯順序的一部份，繼承的子類別需要實作它。
- `usage`: 這個 getter 收集器提供一個簡單的文字收取工作，整合了指令 `name` 以及 `description` 變數內容。
- `ArgResults` 的定義：

```

// 編修 command_runner/lib/src/arguments.dart 檔案內容
// 在該檔案的檔尾後，將加下列類別：
class ArgResults {
  Command? command;
  String? commandArg;
  Map<Option, Object?> options = {};

  // 假如旗標值存在，就回傳 true
  bool flag(String name) {

    // 只確認旗標值，因為旗標值為布林資料型態

    for (var option in options.keys.where(
      (option) => option.type == OptionType.flag,
    )) {
      if (option.name == name) {
        return options[option] as bool;
      }
    }
    return false;
  }

  bool hasOption(String name) {
    return options.keys.any((option) => option.name == name);
  }

  ({Option option, Object? input}) getOption(String name) {
    var mapEntry = options.entries.firstWhere(
      (entry) => entry.key.name == name || entry.key.abbr == name,

```

```
);

    return (option: mapEntry.key, input: mapEntry.value);
  }
}
```

- 此類別表示解析命令列參數的結果。它包含偵測到的命令、該命令的任何參數以及指定選項的對應。
- `flag()`:逐一迭代 `option` 所對應的鍵值，篩選出類型為 `OptionType.flag` 的鍵值。
 - 這裡進行類型轉換（轉換為布林值），因為旗標值是布林值。
- 記錄類型：`getOption()`方法傳回一個記錄類型 (`option: ... , input: ...`)，允許將多個值組合在一起，而無需建立完整的類別。
- Task2: 更新 `CommandRunner` 類別
 - 開啟 `command_runner/lib/src/command_runner_base.dart` 檔案
 - 更新 `CommandRunner` 類別如下：

```
import 'dart:collection';
import 'dart:io';
import 'arguments.dart';

class CommandRunner {
  final Map<String, Command> _commands = <String, Command>{};

  UnmodifiableSetView<Command> get commands =>
    UnmodifiableSetView<Command>(<Command>{..._commands.values});

  Future<void> run(List<String> input) async {
    final ArgResults results = parse(input);
    if (results.command != null) {
      Object? output = await results.command!.run(results);
      print(output.toString());
    }
  }

  void addCommand(Command command) {
    // 錯誤處理：指令名稱不能有衝突
    _commands[command.name] = command;
    command.runner = this;
  }

  ArgResults parse(List<String> input) {
    var results = ArgResults();
    results.command = _commands[input.first];
    return results;
  }

  // 傳回可執行的 usage
  // 如果不想利用 HelpCommand，則應覆寫該方法，或是列出 usage 其它的表示內容

  String get usage {
    final exeFile = Platform.script.path.split('/').last;
    return 'Usage: dart bin/$exeFile <command> [commandArg?] [...options?]
```

```
}
}
```

- 更新後的類別採用新的物件導向結構。
 - 在 `commands` 收集器(getter)中，使用不定長度運算子 (...) 將 `_commands` 對應的值，匯入到一個新集合中。
 - 這方式確保傳回值是副本，防止外部程式碼修改內部資料。
 - 在 `run()` 方法中，`results.command!.run(...)` 使用非空值斷言運算子 (!) 來告訴 Dart 分析器，確認 `results.command` 是否不為空。
 - 這樣做是安全的，因為程式已經在前面的 `if` 語法中檢查過它是否為空。
 - 更重要的關鍵細節：
 - `_commands` 對應：一個私有對應，將指令名稱與其特定的 `Command` 物件實例關聯起來。其值透過 `UnmodifiableSetView` 方法公開。
 - `addCommand()`：註冊一個指令並將此執行器指派給該指令的 `runner` 屬性。這實現了 `Command` 類別中先前做出的延遲初始化承諾。
 - `parse()` 和 `run()`：評估使用者的輸入，從對應中辨識符合的指令，並使用 `await` 呼叫指令的 `run()` 方法。
- 開啟 `command_runner/lib/command_runner.dart` 檔案，輸入程式如下：

```
library;

export 'src/arguments.dart';
export 'src/command_runner_base.dart';
export 'src/help_command.dart';
```

- 以上程式，即可依賴 `command_runner` 的軟體套件就可以存取 `arguments.dart`、`command_runner_base.dart` 和 `help_command.dart` 檔案。
- Task3: 建立 `HelpCommand` 協助說明
 - 建立 `command_runner/lib/src/help_command.dart` 檔案
 - 在 `help_command.dart` 檔案內，建立一 `HelpCommand` 類別，繼承自 `Command` 類別：

```
import 'dart:async';

import 'arguments.dart';

// 列出程式可用的參數項目與使用說明
//

class HelpCommand extends Command {
  HelpCommand() {
    addFlag(
      'verbose',
      abbr: 'v',
      help: '當該項目值為 true 時，將列出每一項可用的參數值。',
    );
    addOption(
      'command',
      abbr: 'c',
      help:
        "當指令略過任何參數值時，將列出該指令的使用方式說明。",
    );
  }
}
```



```

    }
    @override
    String get name => 'help';

    @override
    String get description => '列出本指令的作用說明。';

    @override
    String? get help => '列出本指令的使用方式。';

    @override
    FutureOr<Object?> run(ArgResults args) async {
      var usage = runner.usage;
      for (var command in runner.commands) {
        usage += '\n ${command.usage}';
      }

      return usage;
    }
  }
}

```

- HelpCommand 類別展現出繼承的優勢。
 - 它使用父類別的方法來設定選項，重寫抽象的 run 方法，並存取執行器狀態來產生使用說明。
- Task4: 更新 cli.dart 以使用新的 CommandRunner
 - 開啟 cli/bin/cli.dart 檔案
 - 修改 cli.dart 檔案，使用新版的 CommandRunner 以及 HelpCommand 類別：

```

(以上省略....)
import 'package:command_runner/command_runner.dart';

const version = '0.0.1';

void main(List<String> arguments) {
  var commandRunner = CommandRunner()..addCommand(HelpCommand());
  commandRunner.run(arguments);
}
(以下省略....)

```

- 這段程式碼建立一個 CommandRunner 實例
 - 使用級聯 (cascade) 方法 (...addCommand) 將 HelpCommand 加入到該實例中
 - 該方法允許在建立物件後直接呼叫物件的方法，然後使用命令列參數執行命令執行器。
- Task5: 執行程式
 - 開啟終端機介面，切換至 cli 目錄下
 - 執行下列指令：

```
dart run bin/cli.dart help
```

- 應該可以看見下列訊息，表示程式有正常運作：

```
Usage: dart bin/cli.dart <command> [commandArg?] [...options?]  
help: Prints usage information to the command line.
```

參考文獻

- [Dart 官方教學文件](#)

Generated by [aglio](#) on 30 Apr 2026